

VERİ MADENCİLİĞİ ÖDEVİ

KREDİ KARTI SAHTECİLİĞİ TESPİTİ

Yiğit Akın
2023280086

Kredi Kartı Sahtekarlığı (Fraud) Tespiti

Bu çalışmada kredi kartı işlemlerinin sahte (fraud) olup olmadığını tespit etmek için

- Random Forest

- XGBoost

- Yapay Sinir Ağı (MLP)

modelleri kullanılmıştır.

Veri seti:

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score

from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
```

Başlamadan önce modeller için ihtiyacımız olan kütüphaneler eklendi.

```
df = pd.read_csv("creditcard.csv")
df.head()
```

	Time	V1	V2	V3	V4	V5
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193

5 rows × 31 columns

Verisetimiz okundu ve tablo şekline getirildi. Veri seti 284,807 kredi kartı işleminden oluşmaktadır.

V1-V28: PCA ile dönüştürülmüş özellikler

Amount: İşlem tutarı

Class: Hedef değişken

0: Normal işlem

1: Sahte (Fraud) işlem

Tablonun devamı jupyter dosyasında mevcut.

```
df['Class'].value_counts()
```

```
Class
0      284315
1         492
Name: count, dtype: int64
```

Veri seti ciddi derecede dengesizdir. Bu nedenle değerlendirme metrikleri olarak Precision, Recall, F1-score ve ROC-AUC kullanılmıştır.

```
X = df.drop("Class", axis=1)
y = df["Class"]
```

Model girdileri (X) ile tahmin edilecek hedef değişken (y) birbirinden ayrılmıştır.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Özellikler, farklı ölçeklerde olmalarından kaynaklı yanlılığı önlemek amacıyla standartlaştırılmıştır.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Veri seti, sınıf dağılımı korunacak şekilde eğitim ve test kümelerine ayrılmıştır.

```
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

```
print("Random Forest Results")
print(classification_report(y_test, y_pred_rf))

rf_auc = roc_auc_score(y_test, rf.predict_proba(X_test)[: , 1])
print("ROC-AUC:", rf_auc)
```

```
Random Forest Results
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.96	0.74	0.84	98
accuracy			1.00	56962
macro avg	0.98	0.87	0.92	56962
weighted avg	1.00	1.00	1.00	56962

ROC-AUC: 0.9529092148254912

```
xgb = XGBClassifier(
    scale_pos_weight=(y_train == 0).sum() / (y_train == 1).sum(),
    eval_metric='logloss',
    random_state=42
)

xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)
```

```
print("XGBoost Results")
print(classification_report(y_test, y_pred_xgb))

xgb_auc = roc_auc_score(y_test, xgb.predict_proba(X_test)[: , 1])
print("ROC-AUC:", xgb_auc)
```

```
XGBoost Results
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.88	0.84	0.86	98
accuracy			1.00	56962
macro avg	0.94	0.92	0.93	56962
weighted avg	1.00	1.00	1.00	56962

ROC-AUC: 0.9682378578893572

Random Forest modeli, birden fazla karar ağacının çıktısını birleştirerek sınıflandırma yapan topluluk (ensemble) tabanlı bir yöntemdir.

Modelin oluşturacağı karar ağacı sayısı 100 olarak belirlenerek modelin genelleme yeteneği artırılmıştır.

Deneylerin tekrarlanabilirliğini sağlamak amacıyla rastgelelik sabitlenmiştir.

Dengesiz veri yapısını dengelemek için azınlık sınıfına (fraud) daha fazla ağırlık verilmiştir.

XGBoost modeli, gradient boosting yaklaşımıyla ardışık olarak hataları minimize eden güçlü bir sınıflandırma algoritmasıdır.

Dengesiz veri yapısını dengelemek amacıyla azınlık sınıfının (fraud) modele daha fazla etki etmesi sağlanmıştır.

Modelin eğitim sürecinde hata ölçütü olarak logaritmik kayıp (log loss) kullanılmıştır.

Deneylerin tekrarlanabilir olması için rastgelelik sabitlenmiştir.

```

mlp = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation='relu',
    solver='adam',
    max_iter=50,
    random_state=42
)

mlp.fit(X_train, y_train)
y_pred_mlp = mlp.predict(X_test)

print("Neural Network (MLP) Results")
print(classification_report(y_test, y_pred_mlp))

mlp_auc = roc_auc_score(y_test, mlp.predict_proba(X_test)[:, 1])
print("ROC-AUC:", mlp_auc)

```

Neural Network (MLP) Results				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.82	0.83	0.82	98
accuracy			1.00	56962
macro avg	0.91	0.91	0.91	56962
weighted avg	1.00	1.00	1.00	56962
ROC-AUC: 0.9754798057377143				

MLP (Multi-Layer Perceptron) modeli, birden fazla gizli katman ve nöronlardan oluşan yapay sinir ağı tabanlı bir yöntem olup, doğrusal olmayan ilişkileri öğrenerek sınıflandırma işlemini gerçekleştirmektedir.

Model, iki gizli katman (32 ve 16 nöron) ile oluşturularak karmaşık örüntülerin öğrenilmesi hedeflenmiş, ReLU aktivasyon fonksiyonu ve Adam optimizasyon algoritması kullanılmıştır. Modelin eğitimi 50 iterasyon ile sınırlandırılmış ve deneylerin tekrarlanabilirliğini sağlamak amacıyla rastgelelik sabitlenmiştir.

Modeller, eğitim verisi kullanarak sahte ve normal işlemleri ayırt etmeyi öğrenmiştir. Eğitilen model kullanılarak test verisi üzerindeki sınıflar tahmin edilmiştir. Modellerin precision, recall ve f1-score gibi performans metrikleri hesaplanmıştır. Modellerin sahte ve normal işlemleri ayırt etme başarısı ROC-AUC metriği ile ölçülmüştür.

```

results_df = pd.DataFrame({
    "Model": ["Random Forest", "XGBoost", "Yapay Sinir Ağı (MLP)"],
    "ROC-AUC": [rf_auc, xgb_auc, mlp_auc]
})

results_df

```

	Model	ROC-AUC
0	Random Forest	0.952909
1	XGBoost	0.968238
2	Yapay Sinir Ağı (MLP)	0.975480

Modellerin ROC-AUC değerlerini gösteren tablo.

Bu alıřmada  farklı makine ğrenmesi yöntemi kredi kartı sahtekarlığını tespit etmek için karşılaştırılmıştır.

Dengesiz veri yapısı nedeniyle ROC-AUC ve Recall metrikleri değerdendirmede kritik rol oynamıştır.

Elde edilen sonuçlara göre Yapay Sinir Ağı (MLP) modeli, ROC-AUC metriğine göre en yüksek ayırt etme performansını göstermiştir.

XGBoost modeli MLP'ye oldukça yakın bir performans sergilemiş, Random Forest modeli ise diğer yöntemlere kıyasla daha düşük ancak tatmin edici sonuçlar üretmiştir.